would mean another call for each bet. For a game like the Champions League final, that could be millions of additional calls. Whether one approach is better cannot be determined without more context.

The settlement is partially implemented here with a projection that reads from the market journal. When such a projection generates an event that corresponds to `Market.Closed`, a listener can query the bets of that market and send the `Settle` message to each bet. Remember that the projection is implemented, but the listener is not. Since this listener has some information about the market, it can query the market's odds and embed that information in the `Settle` message.

NOTE You may wonder how the listener knows all the bets on a market. This information can be found in another projection, this time of bets. Each time a bet is opened, its identifier can be stored in a database table together with the market's identifier. Both projections are implemented in this book but are not discussed until chapter 15.

Listing 12.12 shows how the `bet` actor processes the `Settle` message. It uses a helper method that lets you `ask` with more parameters than `replyTo`. The bet must pass its stake to the wallet, and for this, you have `auxCreateRequest(stake: Int) (replyTo: ...)`. If the market responds in time, the request is considered successful, and the bet is closed. If, on the other hand, the market does not respond, the bet is ended with a `FailedState`. An `ask` is used here instead of a `tell` to leave a trail as to why the money was not paid. This information can be queried later with a projection: for example, to give the operator of the betting house the necessary information to solve the problem.